

Proper Notes Format

◆ WHAT SPRING BOOT ACTUALLY DOES BEHIND THE SCENES

Until now, we have only written code.

Now let's understand what Spring actually does when the application starts and when a user sends a request.

This is one of the most important concepts because it explains the complete flow of your project.

Code You Wrote

```
@RestController
@RequestMapping("/email")
public class EmailController {
```

and

```
@PostMapping("/analyze")
public ResponseEntity<EmailResponseDTO> analyze(...) {
```

At first glance it looks simple.

But Spring performs a lot of work behind the scenes.

◆ STEP 1 : SPRING APPLICATION STARTS

When you run:

```
@SpringBootApplication
public class SmartEmailThreatDetectionSystemApplication {

    public static void main(String[] args) {
        SpringApplication.run(...);
    }

}
```

the JVM starts your application.

What does SpringApplication.run() do?

```
SpringApplication.run(...)
```

starts Spring Boot.

Think of it as turning on the entire Spring framework.

What happens after startup?

Spring starts scanning your project.

It checks every package and class.

Something like:

```
controller
service
dto
config
repository
```

Spring looks inside all of them.

Then Spring Finds

```
@RestController
```

on your EmailController class.

Spring immediately understands:

```
"This class handles web requests."
```

Spring Registers The Controller

Internally Spring stores something similar to:

```
EmailController
```

inside its application context.

Think of Application Context as Spring's memory.

What is Application Context?

You can think of it as a giant container.

```
Spring Container
├── EmailController
├── EmailService
├── RestTemplate
└── Other Beans
```

Spring keeps all important objects inside this container.

What is a Bean?

Any object managed by Spring is called a Bean.

Examples:

```
EmailController
EmailService
RestTemplate
```

All of these become Beans.

Important Point

You never create:

```
new EmailController()
```

yourself.

Spring creates it automatically.

◆ STEP 2 : SPRING BUILDS A ROUTING TABLE

Spring finds:

```
@RequestMapping("/email")
```

and stores:

```
Controller URL = /email
```

Then Spring finds:

```
@PostMapping("/analyze")
```

and stores:

```
Method URL = /analyze
```

```
HTTP Method = POST
```

Then Spring Combines Them

```
/email
+
/analyze
```

becomes:

```
/email/analyze
```

Spring Creates A Mapping

Internally Spring creates something similar to:

URL	HTTP METHOD	JAVA METHOD
/email/analyze	POST	analyze()

This table is called the **Routing Table**.

Why Is This Needed?

Later when a request arrives:

```
POST /email/analyze
```

Spring must know:

"Which Java method should I execute?"

The routing table answers that question.

◆ STEP 3 : USER SENDS A REQUEST

Suppose Swagger sends:

```
POST http://localhost:8080/email/analyze
```

Request Body:

```
{
  "emailText": "You won a lottery"
}
```

What is Happening Here?

Swagger is acting as the client.

It sends an HTTP request to your Spring Boot application.

Request contains:

```
URL
HTTP Method
Request Body
Headers
```

Example:

```
Method = POST
URL = /email/analyze
Body =
{
  "emailText": "You won a lottery"
}
```

Request Leaves Swagger

```
Swagger
↓
Internet/Network
↓
Spring Boot Application
```

◆ STEP 4 : REQUEST REACHES TOMCAT

Most beginners think:

```
Request
↓
Controller
```

directly.

That is not true.

There is something in between.

That thing is:

```
Tomcat
```

What is Tomcat?

Tomcat is the web server embedded inside Spring Boot.

It listens for requests.

Think of Tomcat as a security guard.

```
User
↓
Security Guard
↓
Office
```

Security Guard:

```
Tomcat
```

Office:

```
Spring Application
```

So Flow Becomes

```
User
↓
Tomcat
↓
Spring
```

Tomcat Receives

```
POST /email/analyze
```

and asks Spring:

"Who handles this URL?"

◆ STEP 5 : SPRING CHECKS THE ROUTING TABLE

Spring now checks the routing table created earlier.

Remember:

URL	METHOD
/email/analyze	POST

Spring searches:

```
POST /email/analyze
```

It finds:

```
@PostMapping("/analyze")
```

inside:

```
EmailController
```

Spring immediately knows:

"Call analyze() method."

Internally:

```
POST /email/analyze
↓
EmailController.analyze()
```

This automatic matching process is called:

```
Request Mapping
```

Why is it Powerful?

You never write:

```
if(url.equals("/email/analyze"))
```

or

```
switch(url)
```

Spring handles everything.

◆ STEP 6 : JSON IS CONVERTED INTO A JAVA OBJECT

Request Body arrives:

```
{  
  "emailText": "You won a lottery"  
}
```

Your method is:

```
public ResponseEntity<EmailResponseDTO>  
analyze(  
    @RequestBody EmailRequestDTO request  
)
```

Spring sees:

```
@RequestBody
```

and understands:

"Convert incoming JSON into an EmailRequestDTO object."

Internally Spring does something similar to:

```
EmailRequestDTO request =  
    new EmailRequestDTO();
```

Then:

```
request.setEmailText(  
    "You won a lottery"  
);
```

Result:

```
request
```

now contains:

```
emailText =  
"You won a lottery"
```

Important

You never wrote:

```
new EmailRequestDTO()
```

Spring created the object automatically.

Why is @RequestBody Important?

Without:

```
@RequestBody
```

Spring would not know that JSON must be converted into an object.

So:

JSON

becomes:

EmailRequestDTO

automatically.

◆ STEP 7 : METHOD EXECUTION BEGINS

Now Spring finally enters your method.

```
public ResponseEntity<EmailResponseDTO>
analyze(
    @RequestBody EmailRequestDTO request
)
```

At this moment:

```
request.getEmailText()
```

returns:

You won a lottery

Meaning:

request

already contains the data sent from Swagger.

Flow So Far

```
Swagger
↓
Tomcat
↓
Spring Routing
↓
@RequestBody
↓
EmailRequestDTO
↓
analyze()
```

This is exactly where your code starts executing.

The next section continues with:

STEP 8 → Service Layer, Flask API, ML Prediction, ResponseEntity, and Returning JSON to Swagger, which completes the full request-response cycle.